

图应用

双联通分量：判定准则

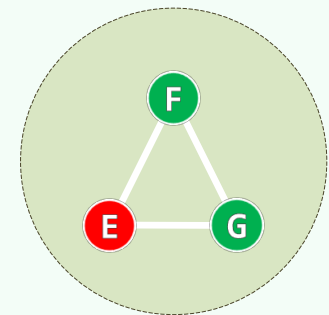
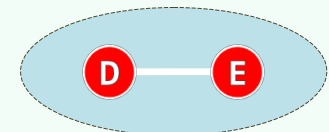
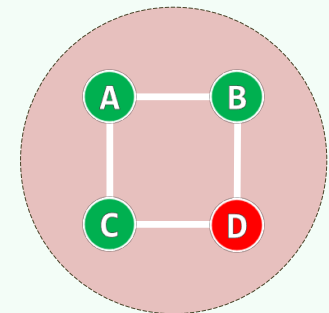
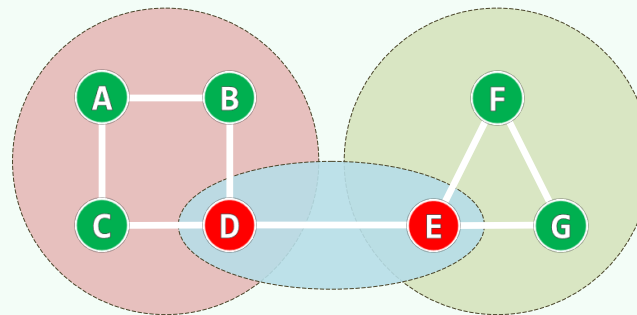
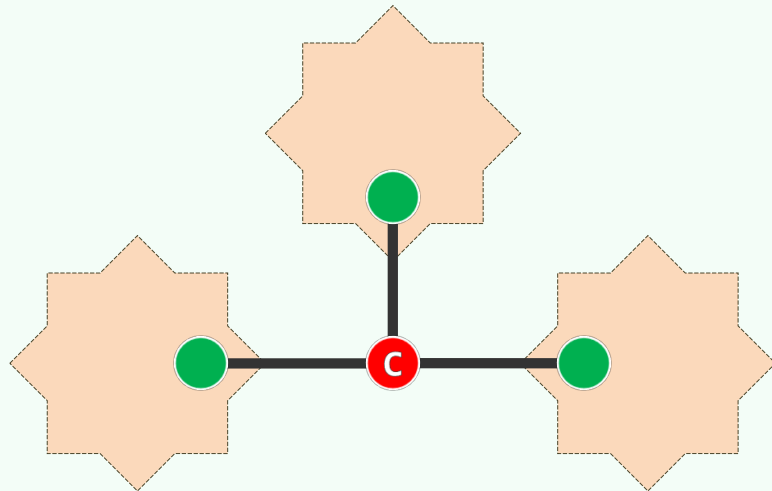
关节点 + 双连通分量

❖ **无向图**的关节点: //articulation point, cut-vertex

其删除之后, 原图的连通分量增多 //connected components

❖ 无关节点的图, 称作双 (重) 连通图 //bi-connectivity

❖ 极大的双连通子图, 称作双连通分量 //Bi-Connected Components



Brute-Force

- ❖ 给定无向图，如何确定各BCC?
- ❖ 先考察简单的版本：如何确定关节点?
- ❖ 蛮力：对每一顶点 v ，通过遍历检查 $G \setminus \{v\}$ 是否连通

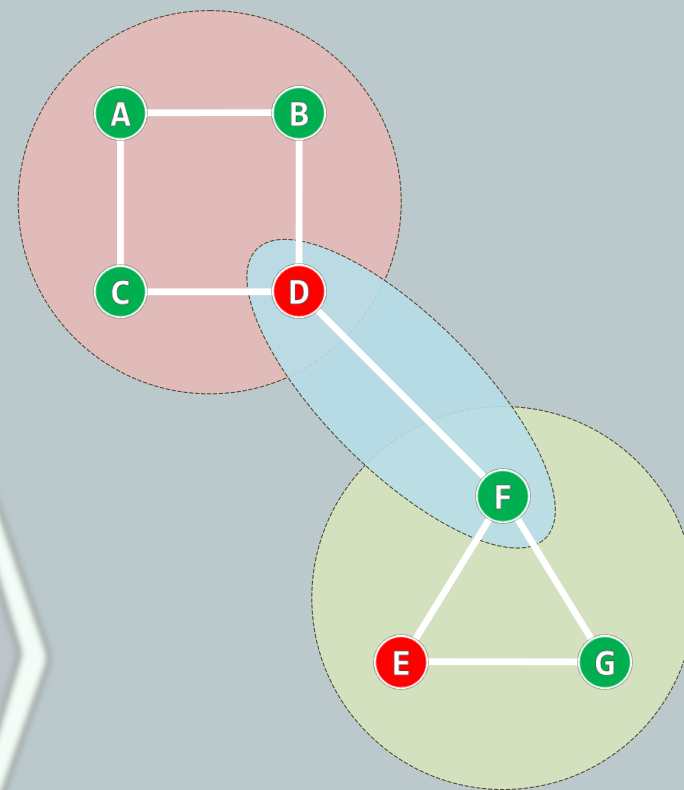
❖ 共需 $O(n * (n + e))$ 时间，太慢!

而且，即便找出关节点，各BCC仍需确定

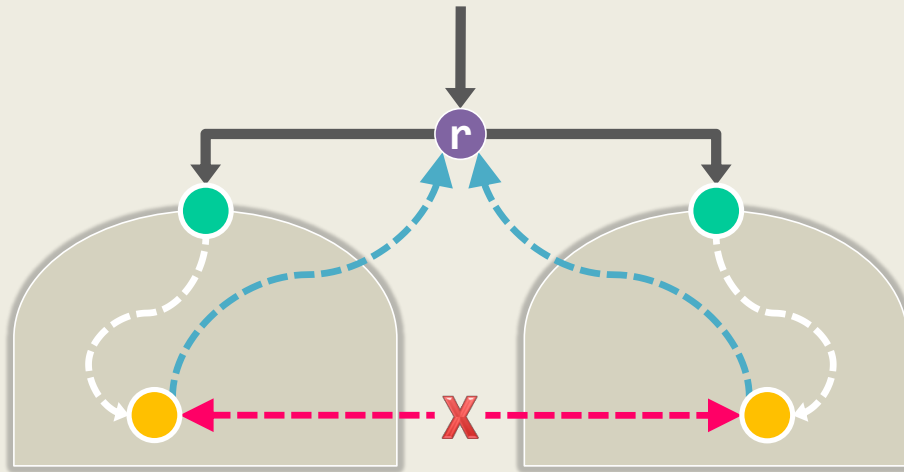
- ❖ 改进：从任一顶点出发，构造DFS树

根据DFS留下的标记，甄别是否关节点

- ❖ 比如，**叶节点**绝不可能是关节点 //为什么?



非叶节点

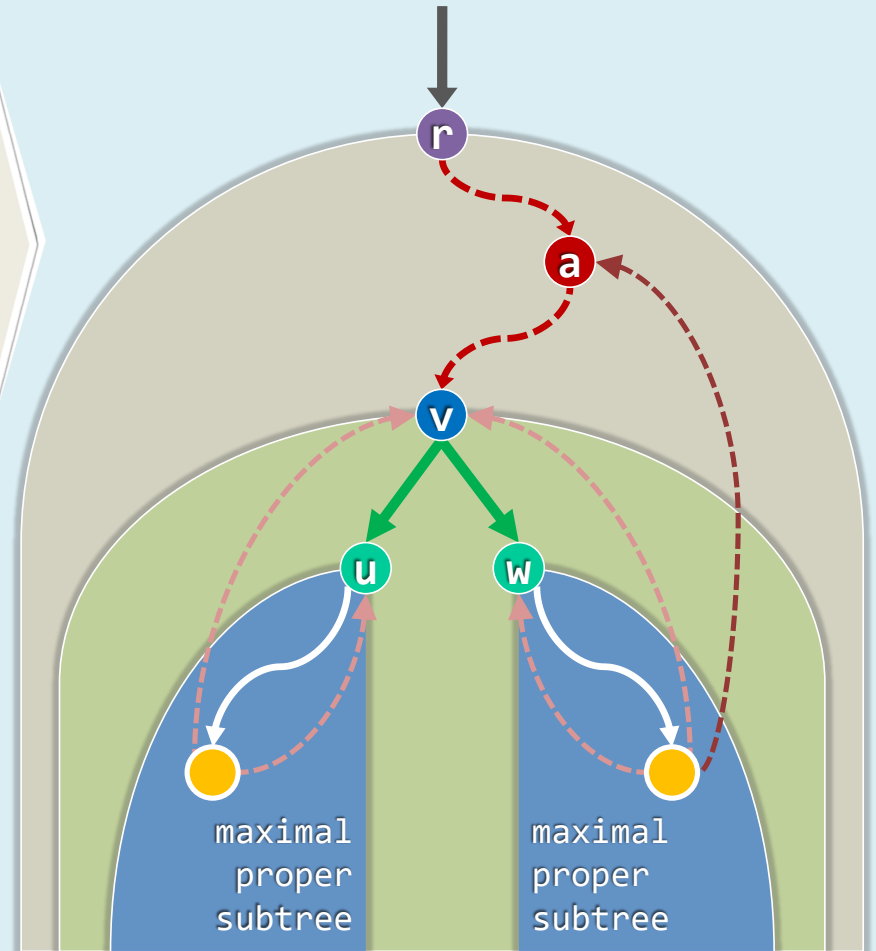


❖ 根 r ：必须至少有2棵子树

❖ 内部节点 v ：

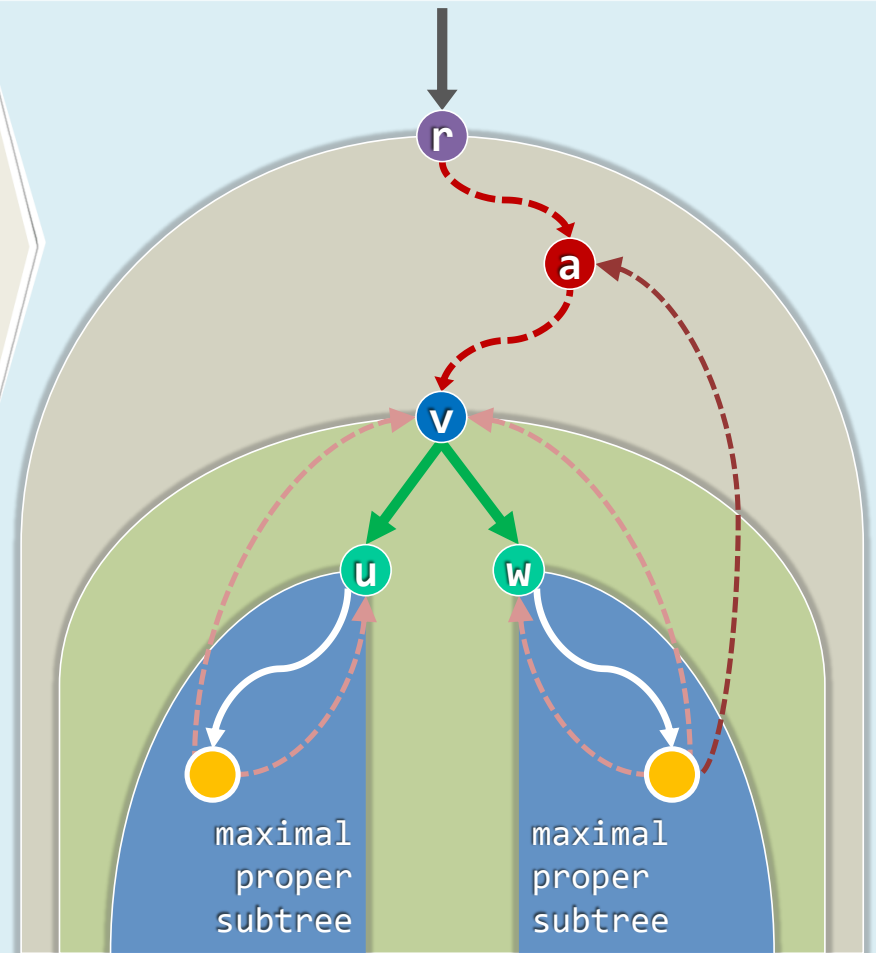
- 有某个孩子 u ，而 $\text{subtree}(u)$ 不能
经由BACKWARD边，联接到 v 的任何真祖先 a

❖ 此时， $\{v\} = \text{BCC}(u) \cap \text{BCC}(\text{parent}(v))$



Highest Connected Ancestor

- ❖ $\text{hca}(v) = \text{subtree}(v)$ 经后向边能抵达的最高祖先
- ❖ 由括号引理: dTime 越小的祖先, 辈份越高
- ❖ DFS过程中, 一旦发现后向边 (v, u)
即取: $\text{hca}(v) = \min(\text{hca}(v), \text{dTime}(u))$
- ❖ DFS(u)完成并返回 v 时
若有: $\text{hca}(u) < \text{dTime}(v)$
即取: $\text{hca}(v) = \min(\text{hca}(v), \text{hca}(u))$
- ❖ 否则, 即可断定: v 系关节点, 且
 $\{v\} + \text{subtree}(u)$ 即为一个BCC



图应用

双联通分量：算法

Graph::BCC()

```
#define hca(x) ( fTime(x) ) //利用此处闲置的fTime

template <typename Tv, typename Te>
void Graph<Tv, Te>::BCC( Rank v, int & clock, Stack<Rank> & S ) {
    hca(v) = dTime(v) = ++clock; status(v) = DISCOVERED; S.push(v);
    for ( Rank u = firstNbr(v); -1 < u; u = nextNbr(v, u) )
        switch ( status(u) )
            { /* ... 视u的状态分别处理 ... */ }
    status(v) = VISITED; //对v的访问结束
}

#undef hca
```

```
switch ( status(u) )
```

```
case UNDISCOVERED:
```

```
    parent(u) = v; type(v, u) = TREE; //拓展树边
```

```
    BCC( u, clock, S ); //从u开始遍历, 返回后...
```

```
    if ( hca(u) < dTime(v) ) //若u经后向边指向v的真祖先
```

```
        hca(v) = min( hca(v), hca(u) ); //则v亦必如此
```

```
    else //否则, 以v为关节点 (u以下即是一个BCC, 且其中顶点此时正集中于栈S的顶部)
```

```
        while ( u != S.pop() ); //弹出当前BCC中 (除v外) 的所有节点
```

```
        //可视需要做进一步处理
```

```
    break;
```



```
switch ( status(u) )
```

```
case DISCOVERED:
```

```
    type(v, u) = BACKWARD;
```

```
    if ( u != parent(v) )
```

```
        hca(v) = min( hca(v), dTime(u) ); //更新hca(v), 越小越高
```

```
    break;
```

```
default: //VISITED (digraphs only)
```

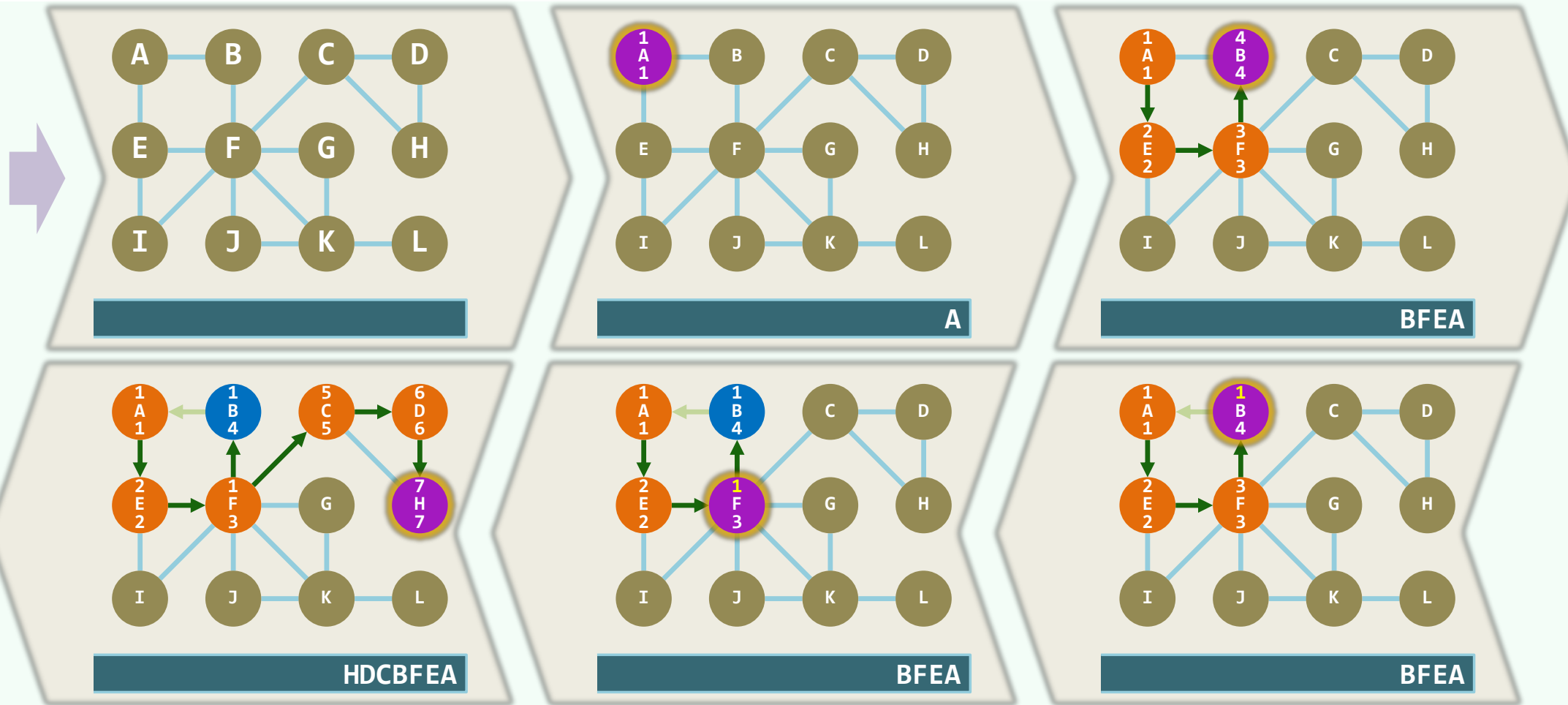
```
    type(v, u) = dTime(v) < dTime(u) ? FORWARD : CROSS;
```

```
    break;
```

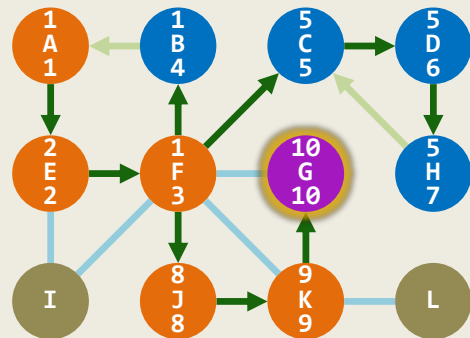
图应用

双联通分量：实例

BCC: Example (1/5)

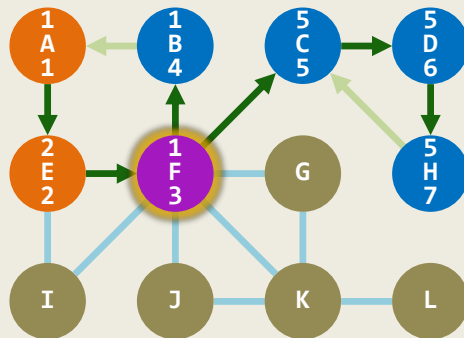


BCC: Example (2/5)



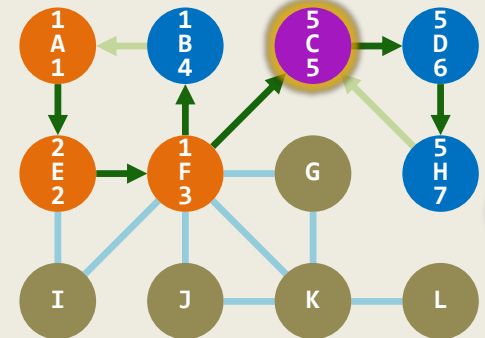
HDC CF

GKJBFEA



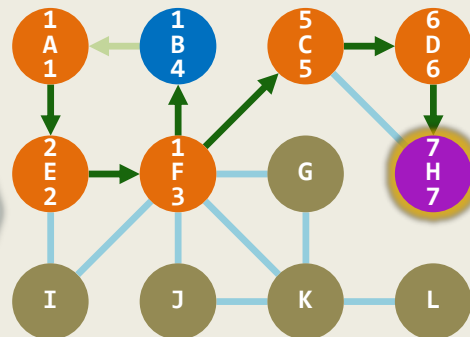
HDC CF

BFEA

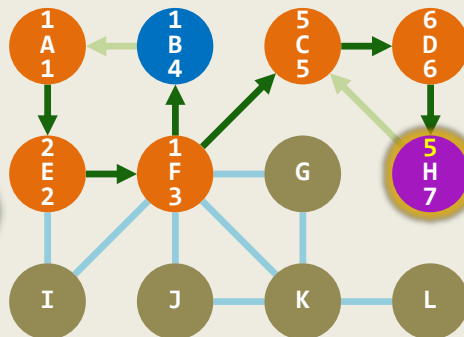


HDC

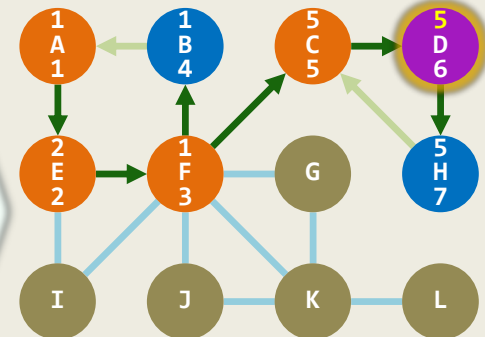
CBFEA



HDCBFEA

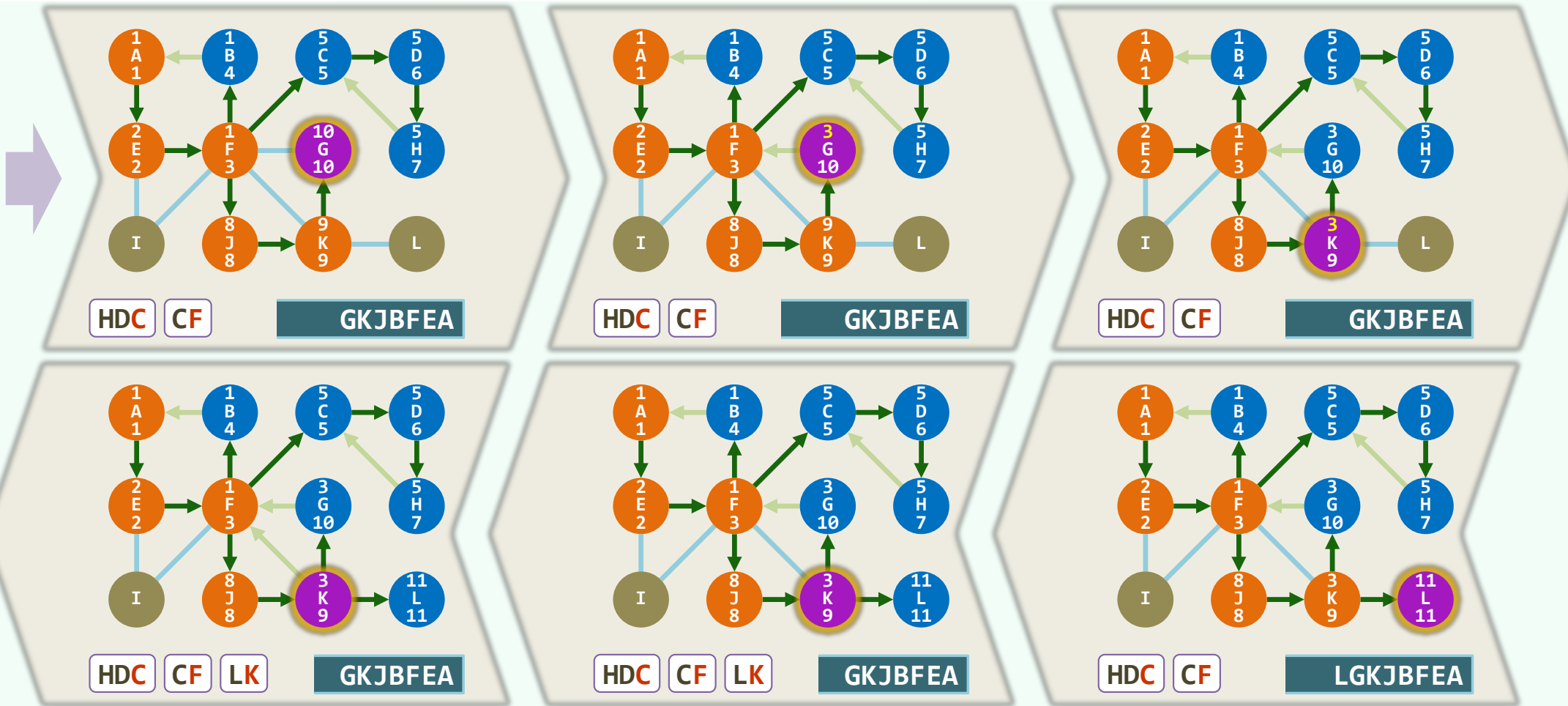


HDCBFEA

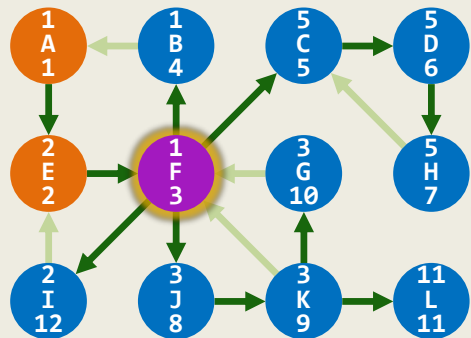


HDCBFEA

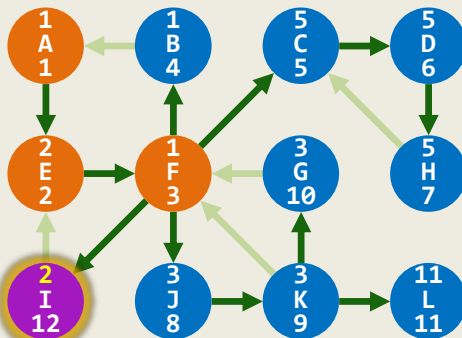
BCC: Example (3/5)



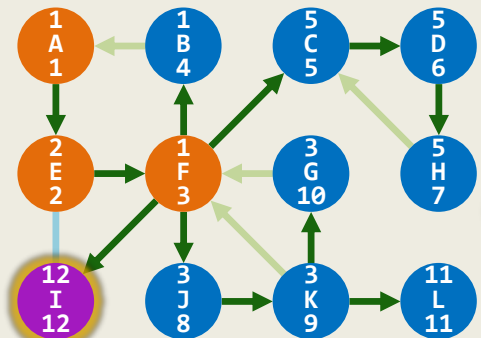
BCC: Example (4/5)



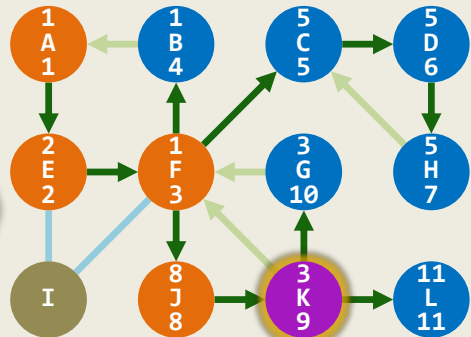
HDC CF LK GKJF IBFEA



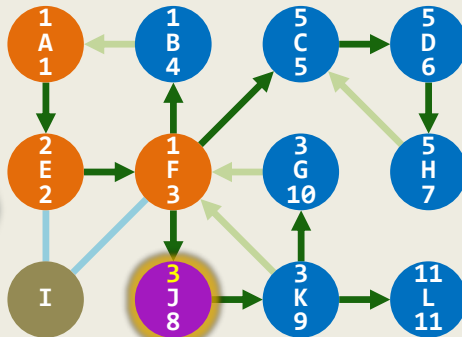
HDC CF LK GKJF IBFEA



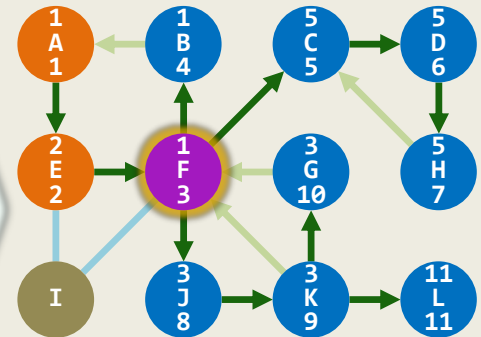
HDC CF LK GKJF IBFEA



HDC CF LK GKJBFEA



HDC CF LK GKJBFEA



HDC CF LK GKJBF E

BCC: Example (5/5)

